DATA STRUCTURES STUDY NOTES

These notes cover the core data structures from a first course. Each section explains what the structure stores, which operations it supports, and how fast those operations are.

1. MEASURING COST WITH BIG-O

Big-O notation describes how the running time or memory of an algorithm grows as the input size n grows. It ignores constant factors and keeps only the fastest-growing term, so 3n + 20 is written O(n) and n^2 + n is written O(n^2).

Common classes, from fastest to slowest: O(1) constant time, O(log n) logarithmic time, O(n) linear time, O(n log n) linearithmic time, O(n^2) quadratic time, and O(2^n) exponential time.

Amortized cost is the average cost per operation over a long sequence of operations. An operation can be expensive once in a while and still have a low amortized cost if the expensive cases are rare enough. Appending to a dynamic array is the standard example: most appends are O(1), an occasional resize is O(n), and the amortized cost of an append is O(1).

2. ARRAYS AND DYNAMIC ARRAYS

An array stores elements in one contiguous block of memory. Because every element has the same size, the address of element i is the start address plus i times the element size, so reading or writing any index takes O(1) time. This is called random access.

Inserting or deleting in the middle of an array takes O(n) time, because every element after that position must shift by one.

A dynamic array grows automatically. When it runs out of space it allocates a new block, usually twice as large, and copies every element across. Doubling the capacity is what makes the amortized cost of append O(1): after a resize to capacity 2c, the next c appends are cheap. Growing by a fixed amount instead, such as 10 extra slots each time, makes the amortized append cost O(n).

Arrays have excellent cache locality. Neighbouring elements sit next to each other in memory, so the processor loads several of them at once, which makes scanning an array much faster in practice than scanning a linked list of the same length.

3. LINKED LISTS

A singly linked list is a chain of nodes. Each node stores a value and a pointer to the next node. The list keeps a pointer to the first node, called the head.

Inserting or deleting at the head takes O(1) time. Inserting after a node you already hold a pointer to also takes O(1) time. Finding the element at position i takes O(n) time, because you must follow i pointers from the head. There is no random access.

A doubly linked list also stores a pointer to the previous node in each node. This allows O(1) deletion of a node when you hold a pointer to it, and traversal in both directions, at the cost of one extra pointer per node.

A common trick for detecting a cycle in a linked list is Floyd’s tortoise and hare algorithm: one pointer moves one step at a time and another moves two steps at a time. If the list has a cycle, the fast pointer eventually meets the slow pointer. It uses O(1) extra memory.

4. STACKS

A stack is a last-in, first-out (LIFO) collection: the most recently added element is the first one removed. The two main operations are push, which adds an element to the top, and pop, which removes the top element. Peek returns the top element without removing it. All three take O(1) time.

A stack can be implemented with a dynamic array, pushing and popping at the end, or with a linked list, pushing and popping at the head.

Uses of stacks include the function call stack, undo history in editors, checking balanced parentheses, evaluating postfix expressions, and depth-first search. To check balanced brackets, push every opening bracket; on a closing bracket, pop and check that it matches. The string is balanced if every pop matches and the stack is empty at the end.

5. QUEUES AND DEQUES

A queue is a first-in, first-out (FIFO) collection: elements leave in the same order they arrived. Enqueue adds an element at the back and dequeue removes the element at the front. Both take O(1) time.

A circular buffer implements a queue with a fixed-size array and two indices, head and tail, that wrap around to the start of the array when they reach the end. This avoids shifting elements when the front element is removed.

A deque, or double-ended queue, allows adding and removing at both ends in O(1) time. Queues are used for breadth-first search, task scheduling, and buffering data between a producer and a consumer.

A priority queue is different: it always removes the element with the highest priority, not the oldest one. It is usually implemented with a heap, covered in section 9.

6. HASH TABLES

A hash table stores key-value pairs and supports insert, lookup and delete in O(1) average time. A hash function turns each key into an integer, and that integer modulo the table size gives the bucket where the pair is stored.

A collision happens when two different keys map to the same bucket. Separate chaining handles collisions by keeping a small list of entries in each bucket. Open addressing handles collisions by probing other buckets in a fixed order until a free one is found; linear probing checks the next bucket, then the one after, and so on.

The load factor is the number of stored entries divided by the number of buckets. As the load factor grows, collisions become more frequent and operations slow down. In these notes, a table is resized to twice as many buckets when the load factor exceeds 0.75, and every entry is rehashed into the new table. With a bad hash function, or an attacker choosing keys, every key can land in the same bucket and operations degrade to O(n) in the worst case.

Hash tables do not keep keys in sorted order. If you need the smallest key or keys in a range, use a balanced binary search tree instead.

7. BINARY SEARCH TREES

A binary search tree (BST) is a binary tree where every node’s key is greater than all keys in its left subtree and smaller than all keys in its right subtree. This ordering property lets search discard half of the remaining tree at each step when the tree is balanced.

Search, insert and delete take O(h) time, where h is the height of the tree. In a balanced tree h is O(log n). If keys are inserted in sorted order, each new key becomes the right child of the previous one, the tree degenerates into a linked list, and h becomes n, so operations take O(n).

An in-order traversal of a BST visits left subtree, node, right subtree, and outputs the keys in sorted order. Pre-order visits the node before its subtrees; post-order visits it after.

Deleting a node with two children replaces its key with its in-order successor, which is the smallest key in its right subtree, and then deletes that successor node, which has at most one child.

8. BALANCED TREES

An AVL tree is a self-balancing BST. For every node, the heights of its left and right subtrees differ by at most one. After an insert or delete, the tree checks this balance factor on the path back to the root and restores it with rotations. A single rotation fixes a left-left or right-right imbalance; a double rotation fixes a left-right or right-left imbalance. Height stays O(log n), so search, insert and delete are all O(log n) in the worst case.

A red-black tree is another self-balancing BST. It colours each node red or black and enforces rules that keep the longest root-to-leaf path at most twice the length of the shortest one. Red-black trees do fewer rotations than AVL trees on insert and delete, while AVL trees are more strictly balanced and slightly faster to search.

A B-tree is a balanced tree whose nodes hold many keys and have many children. Because each node is large, the tree is very shallow, which means few disk reads per lookup. Databases and file systems use B-trees for their indexes.

9. HEAPS

A binary heap is a complete binary tree stored in an array. In a min-heap every parent is smaller than or equal to its children, so the minimum is always at the root. For the node at index i, its children are at indices 2i + 1 and 2i + 2, and its parent is at index (i - 1) / 2 rounded down.

Insert places the new element at the end of the array and sifts it up, swapping with its parent while it is smaller. Extract-min removes the root, moves the last element to the root, and sifts it down, swapping with the smaller child. Both take O(log n) time. Reading the minimum takes O(1).

Building a heap from n unsorted elements by sifting down every non-leaf node from the bottom up takes O(n) time, which is faster than n separate inserts at O(n log n).

Heapsort builds a max-heap and repeatedly moves the maximum to the end of the array. It runs in O(n log n) time in every case and uses O(1) extra memory, but it is not stable.

10. GRAPHS

A graph is a set of vertices connected by edges. Edges can be directed or undirected, and weighted or unweighted.

An adjacency matrix stores a V by V table where entry (u, v) says whether an edge exists. Checking one edge takes O(1) time but the matrix uses O(V^2) memory, which wastes space for sparse graphs. An adjacency list stores, for every vertex, a list of its neighbours. It uses O(V + E) memory and is the usual choice for sparse graphs.

Breadth-first search (BFS) explores the graph level by level using a queue. In an unweighted graph, BFS finds the shortest path, measured in number of edges, from the start vertex to every other vertex. Depth-first search (DFS) goes as deep as possible along each branch before backtracking, using a stack or recursion. DFS is used for cycle detection, topological sorting, and finding connected components. Both BFS and DFS take O(V + E) time with an adjacency list.

A topological sort orders the vertices of a directed acyclic graph so that every edge goes from an earlier vertex to a later one. It exists only if the graph has no cycles.

11. SHORTEST PATHS AND SPANNING TREES

Dijkstra’s algorithm finds the shortest paths from one source vertex in a graph with non-negative edge weights. It keeps a priority queue of vertices ordered by their current best distance, repeatedly removes the closest unfinished vertex, and relaxes its outgoing edges. With a binary heap it runs in O((V + E) log V) time. Dijkstra’s algorithm gives wrong answers when some edge weights are negative.

The Bellman-Ford algorithm handles negative edge weights. It relaxes every edge V - 1 times, which takes O(V E) time, and one more round of relaxation detects a negative cycle if any distance still improves.

A minimum spanning tree connects all vertices of a weighted undirected graph with the smallest total edge weight. Kruskal’s algorithm sorts the edges by weight and adds each edge that does not create a cycle, using a union-find structure to check for cycles. Prim’s algorithm grows the tree from one vertex, always adding the cheapest edge that reaches a new vertex.

12. SORTING

A sorting algorithm is stable if equal elements keep their original relative order. Stability matters when sorting records by one field after they were already sorted by another.

Merge sort splits the array in half, sorts each half recursively, and merges the two sorted halves. It takes O(n log n) time in every case, is stable, and needs O(n) extra memory for merging.

Quicksort picks a pivot, partitions the array into elements smaller and larger than the pivot, and sorts each part recursively. Its average time is O(n log n) but its worst case is O(n^2), which happens when the pivot is always the smallest or largest element, for example when the first element is chosen as pivot on an already sorted array. Choosing a random pivot makes the worst case very unlikely. Quicksort sorts in place and is usually the fastest comparison sort in practice, but it is not stable.

Insertion sort takes O(n^2) time in general but O(n) on nearly sorted input, and it is fast on very small arrays, so practical sorting libraries switch to insertion sort for small subarrays.

No comparison-based sort can beat O(n log n) in the worst case. Counting sort and radix sort avoid comparisons and can sort integers in linear time when the range of values is limited.

13. TRIES AND UNION-FIND

A trie, or prefix tree, stores strings character by character. Each edge is labelled with a character, and each path from the root spells a prefix. Looking up a word of length L takes O(L) time, no matter how many words are stored. Tries are used for autocomplete and spell checking because all words with a given prefix sit in one subtree.

A union-find structure, also called a disjoint-set structure, keeps track of which elements belong to the same group. Find returns the representative of an element’s group, and union merges two groups. With union by rank and path compression, a sequence of operations runs in nearly constant amortized time per operation. Kruskal’s algorithm uses union-find to detect cycles.